Submission 1

2026-10-21

Step 1: Clean the covariate data

```
head(GSE.covar.raw)
```

```
## # A tibble: 6 x 10
##   subjectID geoID      mat_age_yrs        preg_preeclampsia delivery_gestAge_wks
##   <chr>     <chr>      <chr>              <chr>             <chr>
## 1 6053887-M GSM2264346 age_years: 31.066~ pregnancy_condit~ gestation_weeks: 37~
## 2 3064425-M GSM2264347 age_years: 27.794~ pregnancy_condit~ gestation_weeks: 39~
## 3 6094898-M GSM2264348 age_years: 34.318~ pregnancy_condit~ gestation_weeks: 38~
## 4 7050617-M GSM2264349 age_years: 33.426~ pregnancy_condit~ gestation_weeks: 39~
## 5 7016015-M GSM2264350 age_years: 28.279~ pregnancy_condit~ gestation_weeks: 40~
## 6 7054360-M GSM2264351 age_years: 23.501~ pregnancy_condit~ gestation_weeks: 41~
## # i 5 more variables: sampleCollection_gestAge_wks <chr>,
## #   batch_Microarray <chr>, mat_BMI <chr>, mat_vitD <chr>, mat_asthma <chr>
```

```
summary(GSE.covar.raw)
```

```
##      subjectID         geoID        mat_age_yrs  preg_preeclampsia
##  Length   :157   Length   :157   Length   :157   Length   :157
##  N.unique :157   N.unique :157   N.unique :157   N.unique :  2
##  N.blank  :  0   N.blank  :  0   N.blank  :  0   N.blank  :  0
##  Min.nchar:  9   Min.nchar: 10   Min.nchar: 13   Min.nchar: 27
##  Max.nchar:  9   Max.nchar: 10   Max.nchar: 22   Max.nchar: 33
##  delivery_gestAge_wks sampleCollection_gestAge_wks  batch_Microarray
##  Length   :157        Length   :157                 Length   :157
##  N.unique : 44        N.unique : 53                 N.unique :  8
##  N.blank  :  0        N.blank  :  0                 N.blank  :  0
##  Min.nchar: 19        Min.nchar: 32                 Min.nchar: 41
##  Max.nchar: 28        Max.nchar: 41                 Max.nchar: 41
##       mat_BMI         mat_vitD       mat_asthma
##  Length   :157   Length   :157   Length   :157
##  N.unique :140   N.unique :132   N.unique :  2
##  N.blank  :  0   N.blank  :  0   N.blank  :  0
##  Min.nchar: 20   Min.nchar: 45   Min.nchar: 31
##  Max.nchar: 22   Max.nchar: 48   Max.nchar: 31
```

```
# Data Wrangling time! Whoohoo! Seems like there are some extra text in some
# columns. With the help of the trusty mutate function
# we can clean these cells up!
GSE.covar <- GSE.covar.raw %>%
```

```
  mutate(
    mat_age_yrs = str_remove_all(mat_age_yrs,"age_years: "),
    preg_preeclampsia = str_remove_all(preg_preeclampsia, "pregnancy_condition: "),
    delivery_gestAge_wks = str_remove_all(delivery_gestAge_wks, "gestation_weeks: "),
    sampleCollection_gestAge_wks = str_remove_all(sampleCollection_gestAge_wks, "blood_draw_at_gestatio
    batch_Microarray = str_remove_all(batch_Microarray, "microarray_hybridization_date: "),
    mat_BMI = str_remove_all(mat_BMI, "bmi_kg/m2: "),
    mat_vitD = str_remove_all(mat_vitD, "vitamind_25\\(0h\\)_ng/ml_whole_blood_baseline: "),
    mat_asthma = ifelse(str_ends(mat_asthma, "1"), "1", "0"))


# I also noticed all variables are stored as characters, for safe keeping
# I am going to change the ones that look like numeric measure to numeric
GSE.covar <- GSE.covar %>%
  mutate(across(c(mat_age_yrs, delivery_gestAge_wks, sampleCollection_gestAge_wks,
                  mat_BMI, mat_vitD),
                as.numeric))


head(GSE.covar)
```

```
## # A tibble: 6 x 10
##   subjectID geoID      mat_age_yrs preg_preeclampsia delivery_gestAge_wks
##   <chr>     <chr>            <dbl> <chr>                            <dbl>
## 1 6053887-M GSM2264346        31.1 Preeclampsia                      37.4
## 2 3064425-M GSM2264347        27.8 Normal                            39.4
## 3 6094898-M GSM2264348        34.3 Normal                            38.9
## 4 7050617-M GSM2264349        33.4 Normal                            39.1
## 5 7016015-M GSM2264350        28.3 Preeclampsia                      40.1
## 6 7054360-M GSM2264351        23.5 Normal                            41.1
## # i 5 more variables: sampleCollection_gestAge_wks <dbl>,
## #   batch_Microarray <chr>, mat_BMI <dbl>, mat_vitD <dbl>, mat_asthma <chr>
```

```
dim(GSE.covar)
```

```
## [1] 157  10
```

Step 2: Make the categorical variables

```
# Categorical variable #1: preclampsia status (already categories, so we just
# tell R it's a factor and put "Normal" first so it is the reference group).
# Categorical variable #2: vitamin D status, DERIVED from the continuous
# mat_vitD column. Under 20 ng/mL is a commonly used cutoff for deficiency.

GSE.covar <- GSE.covar %>%
  mutate(
    preg_preeclampsia = factor(preg_preeclampsia,
                               levels = c("Normal", "Preeclampsia")),
    vitD_status = factor(ifelse(mat_vitD < 20,
                                "Deficient (<20 ng/mL)",
```

```
                                "Sufficient (>=20 ng/mL)"),
                         levels = c("Deficient (<20 ng/mL)",
                                    "Sufficient (>=20 ng/mL)")))


# Check that the new variable makes sense: group sizes, and that the cutoff
# really split the vitamin D values where we said it would.
table(GSE.covar$preg_preeclampsia, GSE.covar$vitD_status)
```

```
##
##                 Deficient (<20 ng/mL) Sufficient (>=20 ng/mL)
##   Normal                           52                      58
##   Preeclampsia                     26                      21
```

```
GSE.covar %>% group_by(vitD_status) %>% summarise(min = min(mat_vitD), max = max(mat_vitD))
```

```
## # A tibble: 2 x 3
##   vitD_status               min   max
##   <fct>                   <dbl> <dbl>
## 1 Deficient (<20 ng/mL)    5.11  19.9
## 2 Sufficient (>=20 ng/mL) 20.4   80.8
```

Step 3: Clean the gene expression data and attach gene names

```
# The first column of the expression file has a blank header, so we name it
# PROBEID. Probe IDs are just labels, so we store them as text in both tables
# so that R can match them up.

GSE.GE <- GSE.GE.raw %>%
  rename(PROBEID = 1) %>%
  mutate(PROBEID = as.character(PROBEID))

GSE.GN <- GSE.GN.raw %>%
  mutate(PROBEID = as.character(PROBEID))

# Step A: look up the gene name for each probe
# Step B: pivot_longer turns the sideways table (one column per person) into a
#         tall table with one row per gene per person.

GSE.GE <- GSE.GE %>%
  left_join(GSE.GN, by = "PROBEID") %>%
  pivot_longer(cols = -c(PROBEID, GENE),
               names_to = "geoID",
               values_to = "expression")

# Safety checks: every probe got a gene name, and the sample IDs in the
# expression data are exactly the sample IDs in the covariate data.
stopifnot(!any(is.na(GSE.GE$GENE)))
stopifnot(setequal(unique(GSE.GE$geoID), GSE.covar$geoID))

head(GSE.GE)
```

```
## # A tibble: 6 x 4
##   PROBEID GENE    geoID      expression
##   <chr>   <chr>   <chr>           <dbl>
## 1 8089659 CCDC191 GSM2264363       6.18
## 2 8089659 CCDC191 GSM2264379       6.11
## 3 8089659 CCDC191 GSM2264394       6.32
## 4 8089659 CCDC191 GSM2264418       5.91
## 5 8089659 CCDC191 GSM2264449       6.35
## 6 8089659 CCDC191 GSM2264452       5.91
```

Step 4: Pick our genes and build the plotting table

```
# Not every gene in the linker file is in the expression data, so check first!
genes_of_interest <- c("RUNDC3A", "LYN", "IGFBP1")
stopifnot(all(genes_of_interest %in% GSE.GE$GENE))   # stops w an error if a gene is missing

# Each gene should have exactly 157 rows (one probe, one row per person)
GSE.GE %>% filter(GENE %in% genes_of_interest) %>% count(GENE)
```

```
## # A tibble: 3 x 2
##   GENE        n
##   <chr>   <int>
## 1 IGFBP1    157
## 2 LYN       157
## 3 RUNDC3A   157
```

```
# Keep just our genes, then attach each person's covariates using geoID
dat_all <- GSE.GE %>%
  filter(GENE %in% genes_of_interest) %>%
  left_join(GSE.covar, by = "geoID")

# Should be 157 rows: one per person
stopifnot(nrow(dat_all) == length(genes_of_interest) * nrow(GSE.covar))
```

Step 5: Build the plotting function

```
# Inputs:
#   data = long data frame with one row per person per gene. It needs the
#   column GENE, expression, geoID, mat_BMI, preg_Preeclampsia, and vitD_status

#   gene = character vector with the names of 1 or more genes

# Output: prints 3 plots per gene, and (invisibly) returns the plots plus a
# small summary table of the numbers shown on them

plot_gene_expression <- function(data, genes) {
  plots <- list()       # holds every plot so we can reuse them later
  gene_stats <- list() # holds one row of summary numbers per gene
```

```r
  # Loop through each gene and make its 3 plots
  for (g in genes){

    # Keep only this gene's rows
    d <- data %>% filter(GENE == g)


# PLOT 1: Histogram of gene expression
    m <- mean(d$expression); s <- sd(d$expression)

    p1 <- ggplot(d, aes(x = expression)) +
      geom_histogram(bins = 20, fill = "dodgerblue", color = "white") +
      geom_vline(xintercept = m, color = "orange", linewidth = 1, linetype = "dashed") +
      annotate("text", x = Inf, y = Inf, hjust = 1.05, vjust = 1.5, size = 5,
               label = paste0("n = ", nrow(d),
                              "\nMean = ", round(m,2),
                              "\nSD = ", round(s, 2))) +
      scale_y_continuous(expand = expansion(mult = c(0, 0.4))) +
      labs(x = paste(g, "expression level"),
           y = "Number of samples") +
      theme_classic(base_size = 14)

# PLOT 2: Scatterplot of BMI vs. Gener expression

    # Pearson correlation between BMI and expression for this gene
    ct <- cor.test (d$mat_BMI, d$expression)

    # Write very small p-values as "p < 0.001" so they don't up as "p = 0.00"
    p_text <- ifelse(ct$p.value < 0.001, "p < 0.001", sprintf("p = %.2f", ct$p.value))


    p2 <- ggplot(d, aes(x = mat_BMI, y = expression)) +
      geom_point(alpha = 0.7, size = 2, color = "dodgerblue") +
      geom_smooth(method = "lm", formula = y~ x, color = "orange",
                  fill = "orange", alpha = 0.2) +
     annotate("text", x = Inf, y = Inf, hjust = 1.1, vjust = 1.5, size = 5,
         label = paste0("r = ", round(ct$estimate, 2),
                        "\n", p_text,
                        "\nn = ", sum(!is.na(d$mat_BMI)))) +
      scale_y_continuous(expand = expansion(mult = c(0.05, 0.4))) +
      labs(x = expression("Maternal BMI (kg/m"^2*")"),
           y = paste(g, "expression level")) +
      theme_classic(base_size = 14)


## PLOT 3: Boxplot of expression by both categorical variables

      # Linear model with both categorical variables (used for the summary table)
    fit <- lm(expression ~ preg_preeclampsia + vitD_status, data = d)
    co  <- summary(fit)$coefficients

    # Group sizes, with the height for the "n = ..." labels set just above this
    # gene's highest point (the old fixed height of 11.4 only worked for one gene)
```

```
    n_labels <- d %>%
      count(preg_preeclampsia, vitD_status) %>%
      mutate(y = max(d$expression) + 0.05 * diff(range(d$expression)))

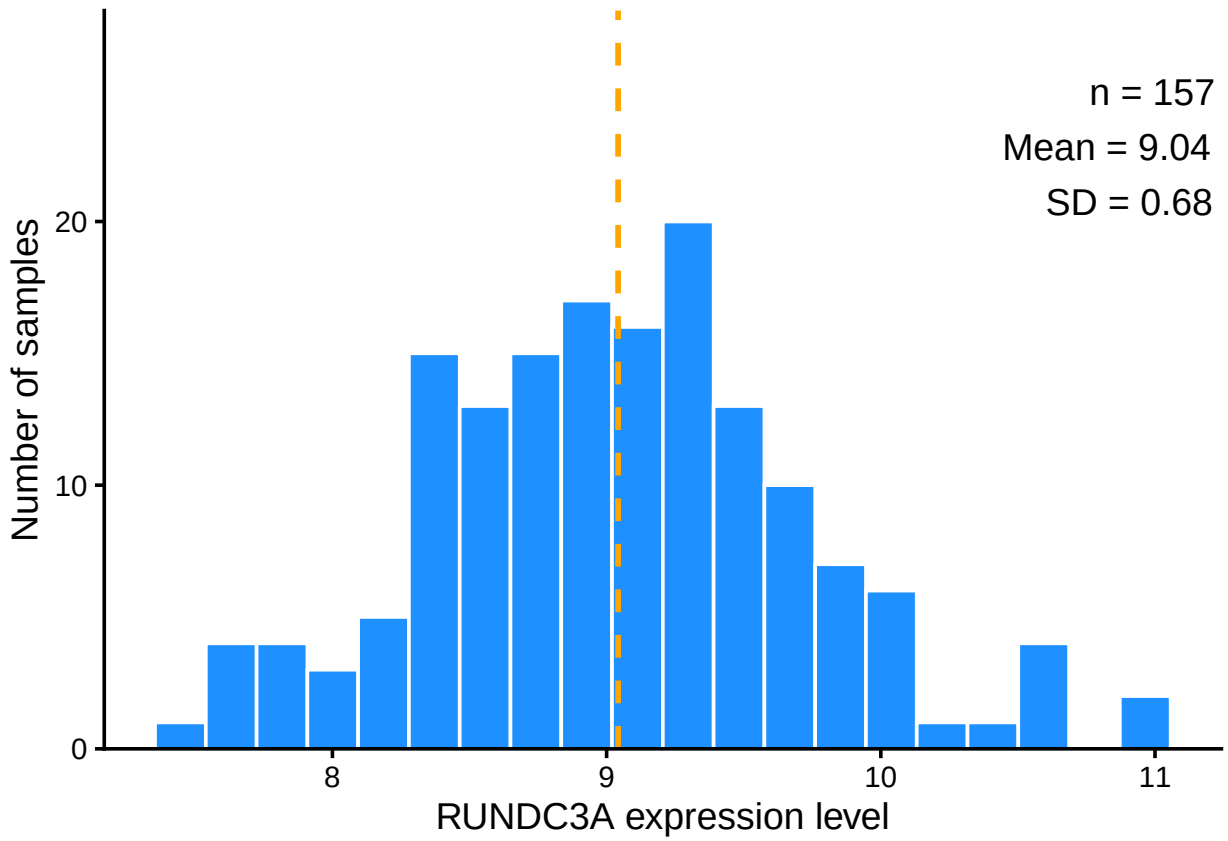

     p3 <- ggplot(d, aes(x = preg_preeclampsia, y = expression, fill = vitD_status)) +
      geom_boxplot(outlier.shape = NA, alpha = 0.7,
                   position = position_dodge(width = 0.8), width = 0.7) +
      geom_point(aes(group = vitD_status),
                 position = position_jitterdodge(jitter.width = 0.15,
                                                 dodge.width = 0.8, seed = 1),
                 alpha = 0.5, size = 1.5, show.legend = FALSE) +
      geom_text(data = n_labels, aes(y = y, label = paste0("n = ", n)),
                position = position_dodge(width = 0.8), size = 4) +
      scale_fill_manual(values = c("orange", "dodgerblue")) +
      labs(x = "Pregnancy outcome",
           y = paste(g, "expression level"),
           fill = "Vitamin D status") +
      theme_classic(base_size = 14)

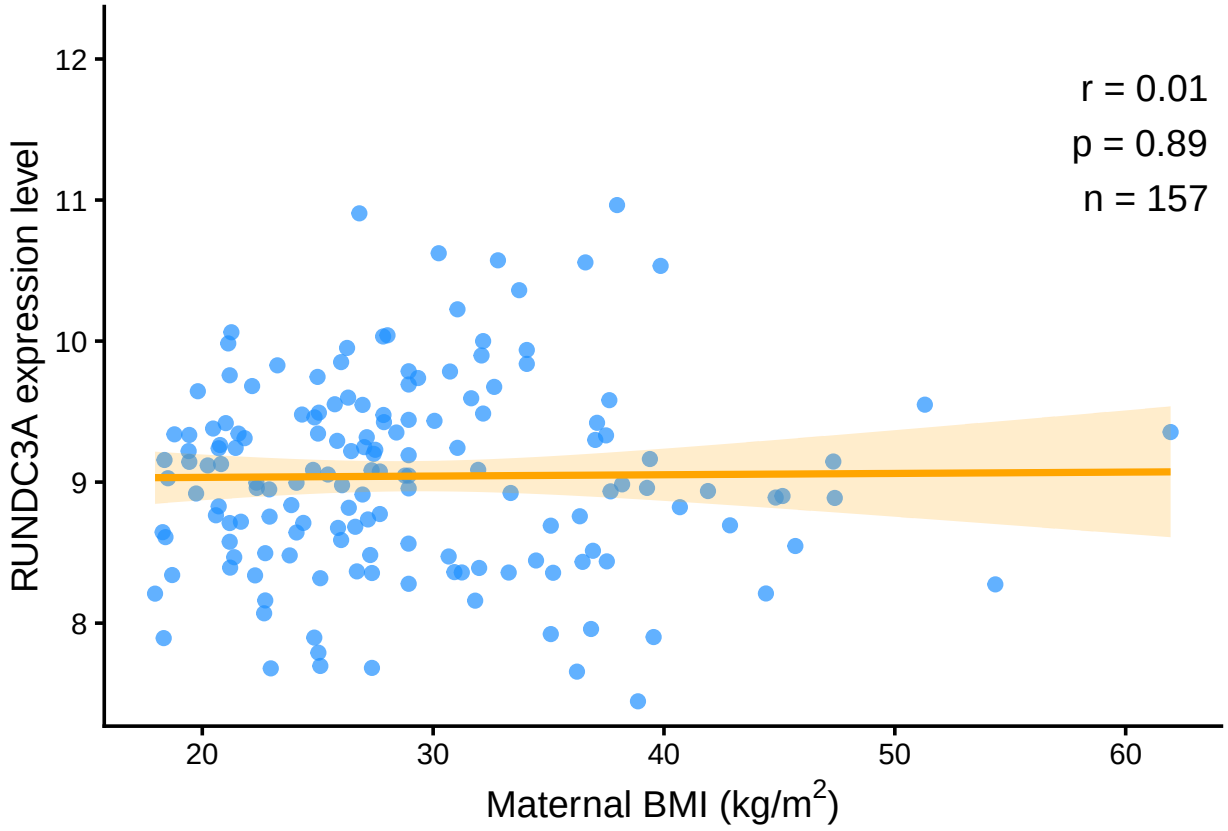
# Show the 3 plots (plots made inside a loop don't show up unless we print them)
    print(p1); print(p2); print(p3)

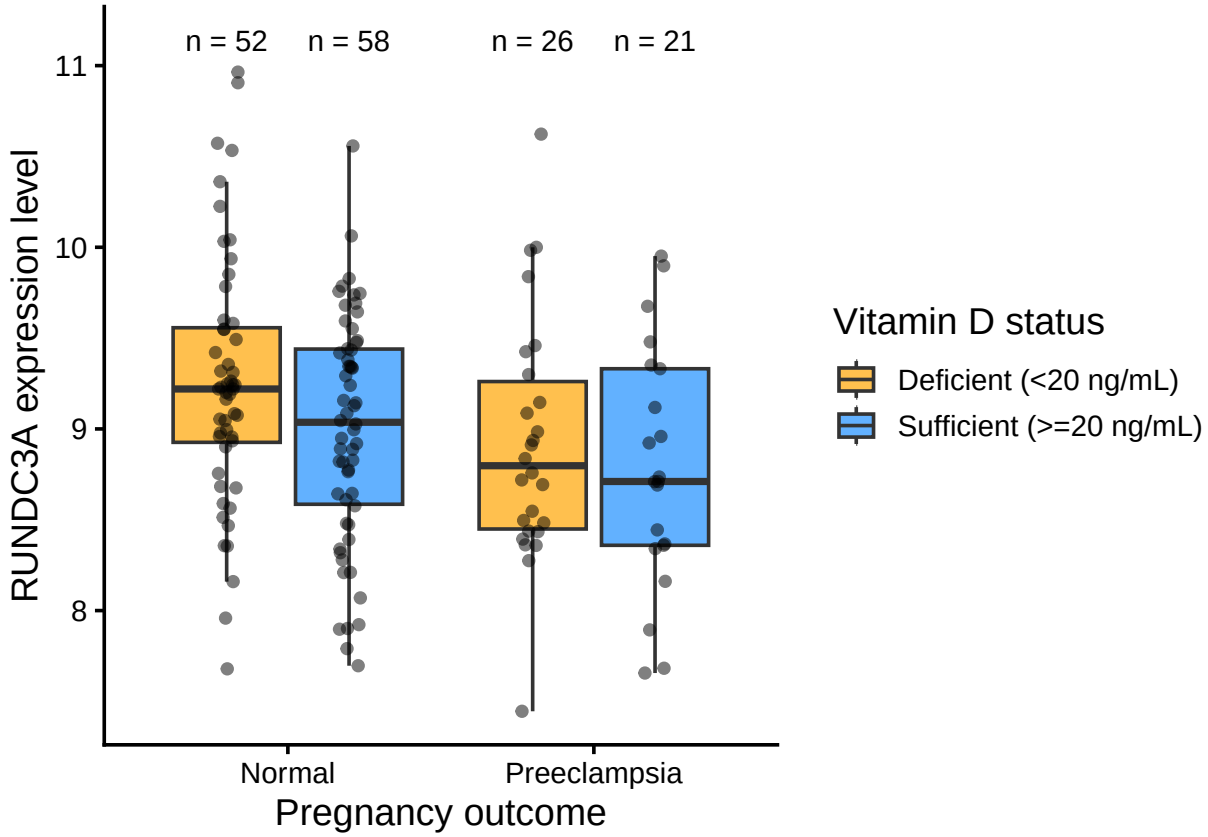
    # Save this gene's plots and its summary numbers
    plots[[g]] <- list(histogram = p1, scatter = p2, boxplot = p3)
    gene_stats[[g]] <- tibble(
      gene = g, mean = m, sd = s,
      r_BMI = unname(ct$estimate), p_BMI = ct$p.value,
      preeclampsia_effect = co[2, "Estimate"], p_preeclampsia = co[2, "Pr(>|t|)"],
      vitD_effect = co[3, "Estimate"], p_vitD = co[3, "Pr(>|t|)"])
  }

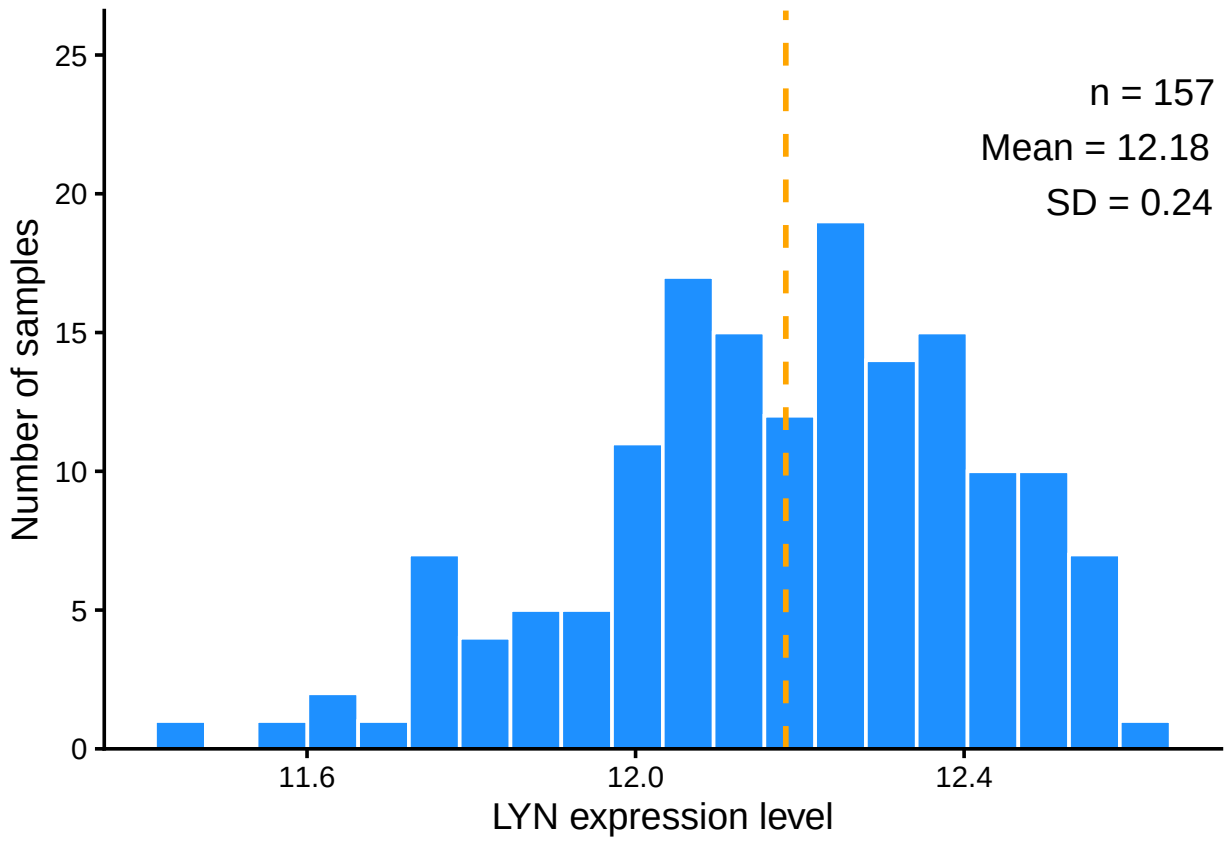
  invisible(list(plots = plots, stats = bind_rows(gene_stats)))
}
```

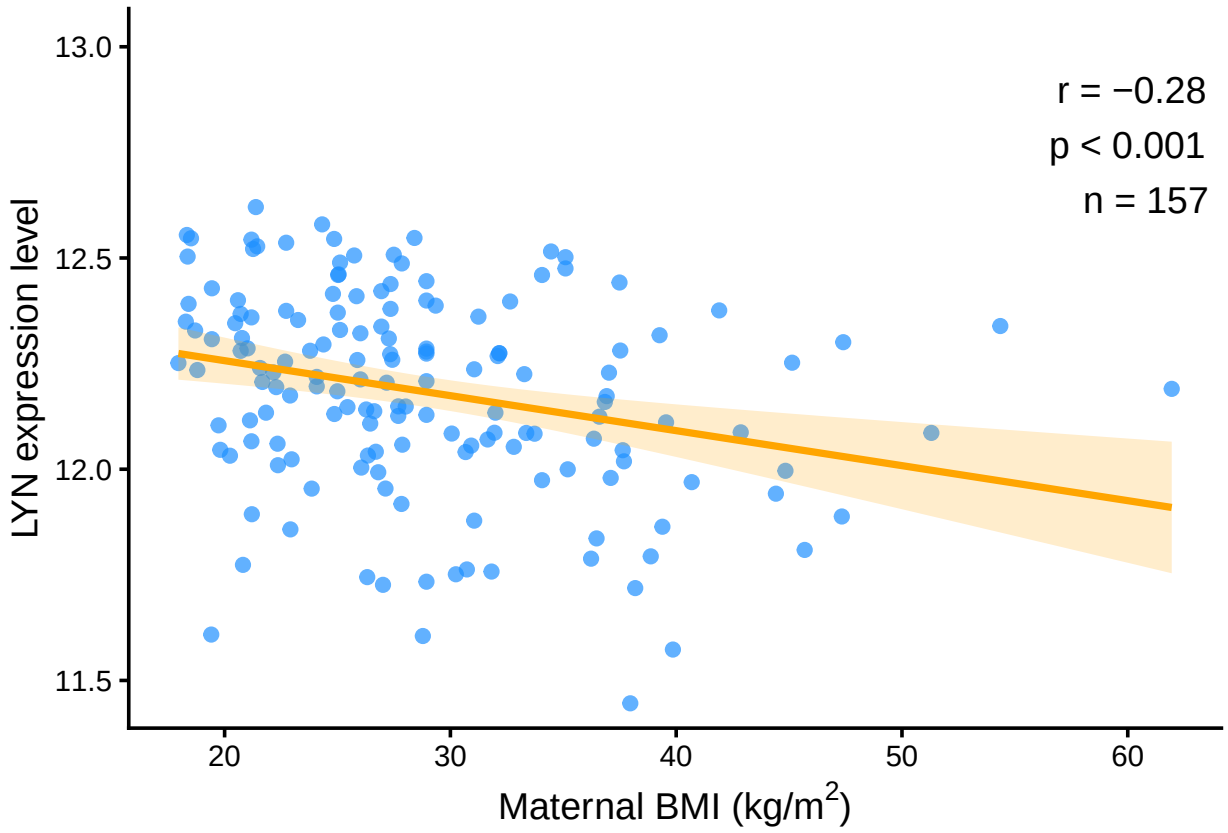
```
gene_results <- plot_gene_expression(dat_all, genes_of_interest)
```

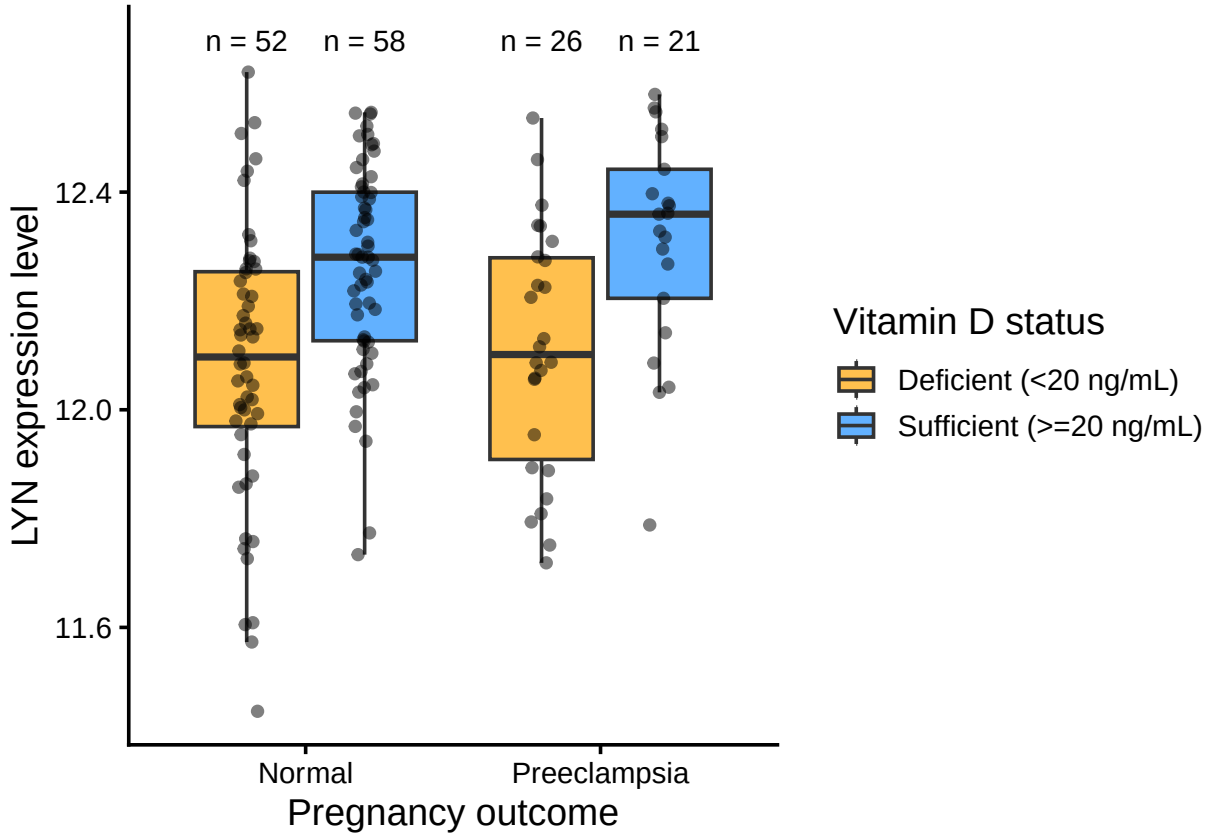

n = 157
Mean = 9.04
SD = 0.68
Number of samples
20
10
0
8
9
10
11
RUNDC3A expression level

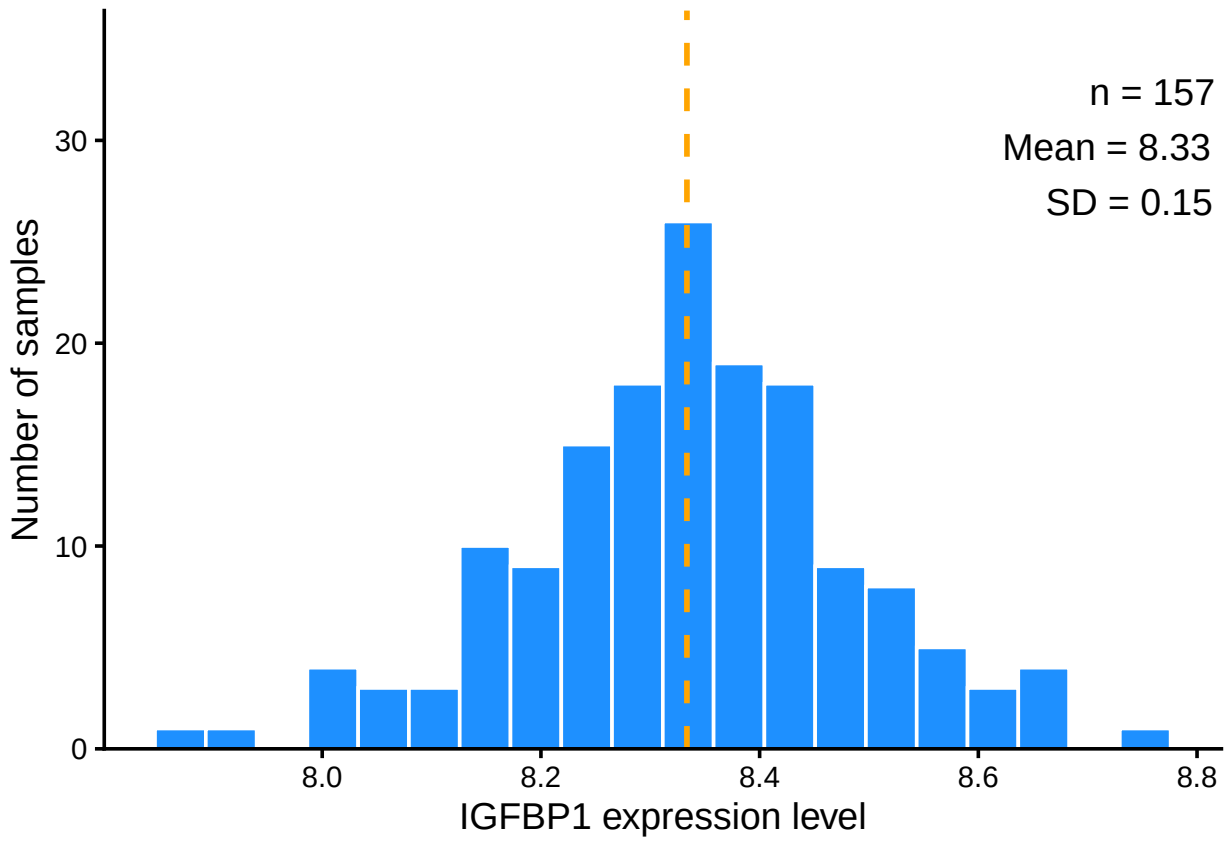

r = 0.01
p = 0.89
n = 157
RUNDC3A expression level
12
11
10
9
8
20
30
40
50
60
Maternal BMI (kg/m2)

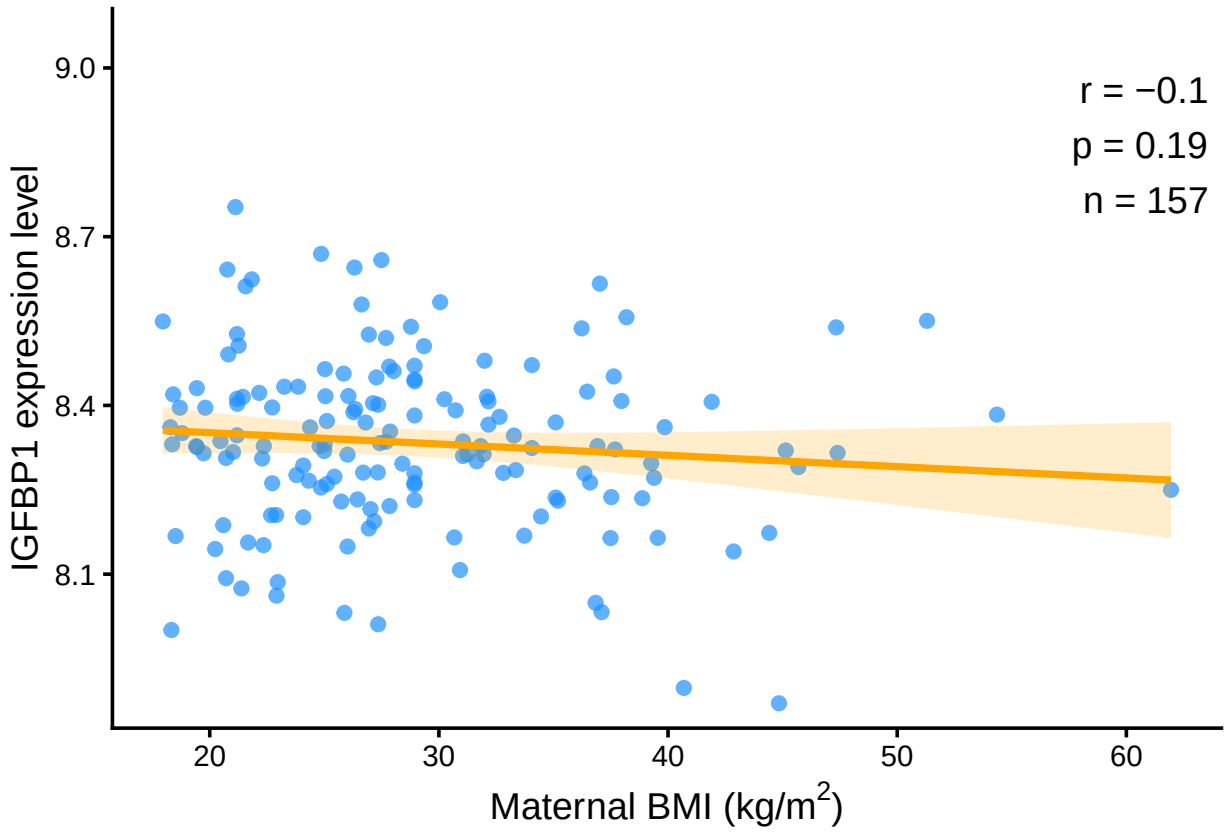

n = 52
n = 58
n = 26
n = 21
11
10
9
8
RUNDC3A expression level
Normal
Preeclampsia
Pregnancy outcome
Vitamin D status
Deficient (<20 ng/mL)
Sufficient (>=20 ng/mL)

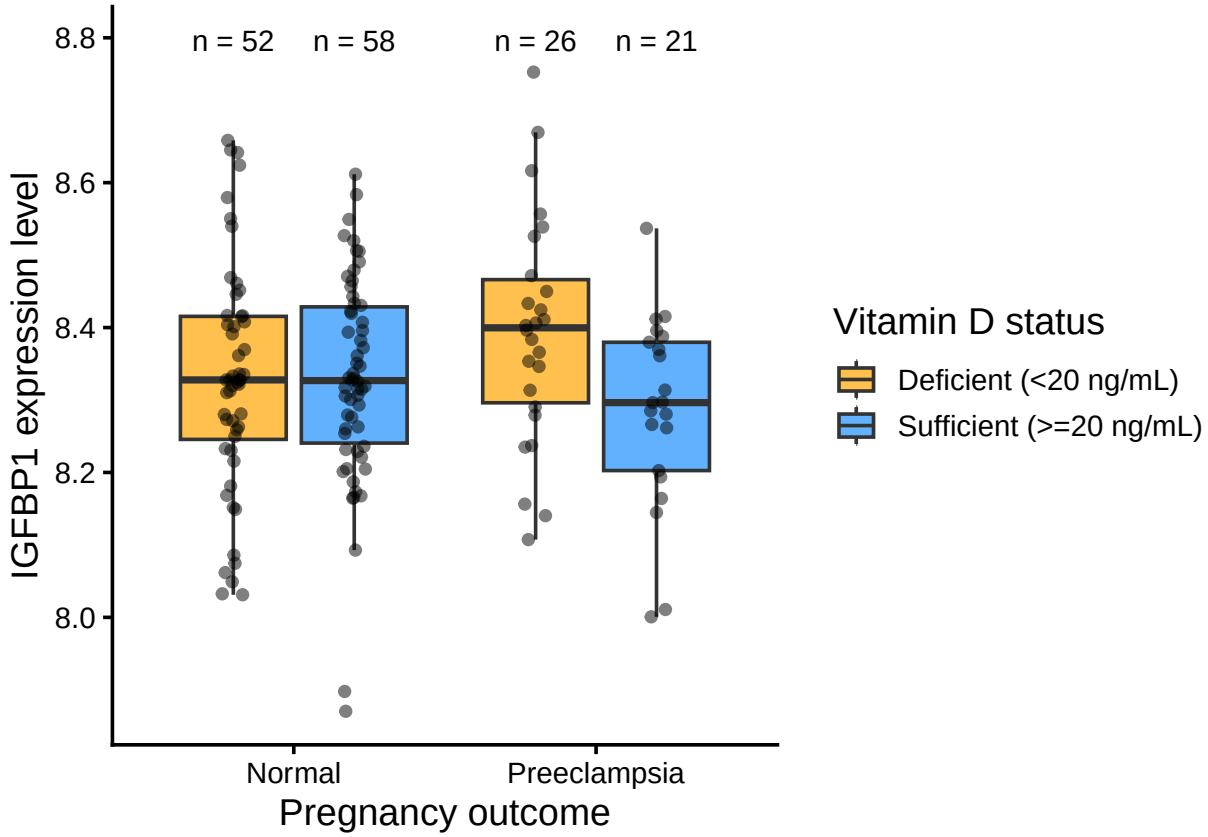

n = 157
Mean = 12.18
SD = 0.24
Number of samples
25
20
15
10
5
0
11.6
12.0
12.4
LYN expression level


r = −0.28
p < 0.001
n = 157
LYN expression level
13.0
12.5
12.0
11.5
20
30
40
50
60
Maternal BMI (kg/m2)


n = 52
n = 58
n = 26
n = 21
12.4
12.0
11.6
LYN expression level
Normal
Preeclampsia
Pregnancy outcome
Vitamin D status
Deficient (<20 ng/mL)
Sufficient (>=20 ng/mL)


n = 157
Mean = 8.33
SD = 0.15
Number of samples
0
10
20
30
8.0
8.2
8.4
8.6
8.8
IGFBP1 expression level


r = −0.1
p = 0.19
n = 157
IGFBP1 expression level
9.0
8.7
8.4
8.1
20
30
40
50
60
Maternal BMI (kg/m2)



ADDITIONAL RESOURCES USED:

- Google & Gemini: to look up commands we learned in class, and extra commands for more advanced arguments.
- Claude (sonnet, medium): to check my work for errors, and help me 'dumb down'/understand specific characteristics in the data and rstudio itself.